

LAPORAN TUGAS 1-4

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Fagil Lola Prihernando (2410651031)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

LATAR BELAKANG

Perkembangan teknologi sistem operasi modern menuntut pemahaman mendalam tentang konsep manajemen proses, manajemen file, shell scripting, dan monitoring sistem. Dalam lingkungan komputasi kontemporer, kemampuan untuk mengembangkan tools yang dapat memantau, mengelola, dan berinteraksi dengan sistem operasi menjadi kompetensi fundamental bagi mahasiswa teknik informatika. Praktikum sistem operasi ini dirancang untuk memberikan pengalaman langsung dalam mengimplementasikan berbagai konsep teoritis ke dalam aplikasi praktis melalui empat tugas utama yang mencakup aspek-aspek kritis dalam pengelolaan sistem. Implementasi process manager, file monitor, custom shell, dan system information tool tidak hanya memperkuat pemahaman konseptual tetapi juga mengembangkan keterampilan pemecahan masalah dalam lingkungan sistem yang sesungguhnya. Melalui pendekatan hands-on ini, mahasiswa dapat memahami kompleksitas dan mekanisme internal sistem operasi secara komprehensif.

RUMUSAN MASALAH

1. Bagaimana mengimplementasikan manajemen proses yang efektif menggunakan fork, pipe, dan shared memory untuk menangani eksekusi paralel multiple processes?
2. Bagaimana membangun sistem monitoring file yang mampu mendeteksi perubahan secara real-time dan mencatat aktivitas filesystem dengan notifikasi yang komprehensif?
3. Bagaimana mengembangkan custom shell dengan fitur advanced seperti multiple pipes, redirection, dan background processes yang stabil tanpa memory leaks?

TUJUAN

1. Mengembangkan process manager yang dapat menangani eksekusi paralel tiga proses anak (faktorial, bilangan prima, dan pembalik string) dengan mekanisme IPC yang tepat.
2. Membuat file monitoring system yang mampu mendeteksi event CREATE, MODIFY, DELETE secara real-time dengan logging dan notifikasi yang andal.
3. Membangun custom shell dengan dukungan multiple pipes, I/O redirection, dan background processes yang memiliki manajemen memori yang optimal.

BAB 2

PEMBAHASAN

2.1 TUGAS 1

Kode pada gambar menunjukkan proses kompilasi dan eksekusi program *process_manager.c*. Saat dikompilasi, muncul satu *warning* karena ketidaksesuaian format pada fungsi printf, yaitu %ld digunakan untuk mencetak nilai bertipe int. Setelah diperbaiki dengan perintah `gcc -o process_manager process_manager.c`, program berhasil dijalankan. Hasilnya menampilkan tiga proses anak: menghitung faktorial, menampilkan bilangan prima, dan membalik string sesuai masukan pengguna.

```
asus@LAPTOP-VRS206DG:~$ cd praktikum_os
asus@LAPTOP-VRS206DG:~/praktikum_os$ gcc process_manager process_manager.c
process_manager.c: In function 'main':
process_manager.c:123:49: warning: format '%ld' expects argument of type 'long int', but argument 4 has type 'int' [-Wformat=]
123 |         printf("\nChild PID: %d | Waktu: %ld.%ld detik\n", w, sec, (int)usec);
    |                                     ^~~~~~
    |                                     |
    |                                 long int
    |                                 |
    |                             %d
    |                             |
    |                         int
/usr/bin/ld: process_manager: stdio: invalid version 3 (max 0)
/usr/bin/ld: process_manager: error adding symbols: bad value
collect2: error: ld returned 1 exit status
asus@LAPTOP-VRS206DG:~/praktikum_os$ ./process_manager
Masukkan angka untuk faktorial: 5
Masukkan batas bilangan prima: 6
Masukkan string untuk dibalik: ollah
\nChild PID: 1633 | Waktu: 0.749 detik\nHasil faktorial: 120\n\nChild PID: 1634 | Waktu: 0.420 detik\nBilangan prima: 2 3 5 \n\nChild
PID: 1635 | Waktu: 0.269 detik\nString terbalik: hallo\nasus@LAPTOP-VRS206DG:~/praktikum_os$
```

2.1.2 Penjelasan Proses Kompleksi dan Eksekusi

1. cd Praktikum_os

- **Fungsi:** Masuk ke direktori bernama praktikum_os.
- **Tujuan:** Agar kamu bisa mengakses file sumber program process_manager.c yang berada di folder tersebut.
- **Penjelasan tambahan:** cd (change directory) digunakan untuk berpindah direktori kerja di terminal Linux.

2. gcc process_manager process_manager.c

- **Fungsi:** Mencoba mengompilasi file `process_manager.c` dan menghasilkan file biner bernama `process_manager`.
- **Kesalahan:**
 - warning: format '%ld' expects argument of type 'long int', but argument 4 has type 'int'
→ Terjadi karena `%ld` digunakan untuk menampilkan long int, tetapi variabel `(int)usec` bertipe int.
 - Harusnya gunakan `%d` jika variabel bertipe int.
- **Error tambahan:**
 - `/usr/bin/ld: process_manager: stdin: invalid version 3 (max 0)`
 - `collect2: error: ld returned 1 exit status`
→ Menunjukkan proses *linking* gagal (biasanya karena kesalahan versi format biner atau parameter output salah).

3. `./process_manager`

- **Fungsi:** Menjalankan file hasil kompilasi bernama `process_manager`.
- **Tanda `./`:** Menunjukkan bahwa file eksekusi berada di direktori saat ini (`.`).
- **Hasil saat dijalankan:**
 - Program meminta tiga input:
 1. Angka untuk faktorial (Masukkan angka untuk faktorial: 5)
 2. Batas bilangan prima (Masukkan batas bilangan prima: 6)
 3. String untuk dibalik (Masukkan string untuk dibalik: ollah)
 - Output yang dihasilkan menunjukkan proses paralel (tiap *child process* memiliki PID berbeda):
 - Child PID: 1633 | Waktu: 0.749 detik | Hasil faktorial: 120
 - Child PID: 1634 | Waktu: 0.420 detik | Bilangan prima: 2 3 5

- Child PID: 1635 | Waktu: 0.269 detik | String terbalik: hallo

4. nano

- **Fungsi:** Membuka teks editor terminal bernama **Nano** untuk membuat atau mengedit file teks (seperti process_manager.c).
- **Contoh penggunaan:**
 - nano process_manager.c
 - kode:

2.2 PRAKTIK TUGAS 2

2.2.1

File Monitor menjelaskan proses pembuatan, kompilasi, dan eksekusi program pemantauan direktori. Tahapan dimulai dengan membuat folder kerja menggunakan mkdir, lalu menulis kode file_monitor.c melalui nano. Setelah dikompilasi dengan gcc, program dijalankan untuk memantau direktori monitor_dir. Saat file dibuat, diubah, atau dihapus, program menampilkan notifikasi event beserta PID. Proses dihentikan menggunakan sinyal SIGTERM atau perintah kill.

Before (sebelum):

```
asus@LAPTOP-VRS206DG:~$ mkdir file_monitor_task
asus@LAPTOP-VRS206DG:~$ cd file_monitor_task
asus@LAPTOP-VRS206DG:~/file_monitor_task$ monitor_dir
monitor_dir: command not found
asus@LAPTOP-VRS206DG:~/file_monitor_task$ mkdir monitor_dir
asus@LAPTOP-VRS206DG:~/file_monitor_task$ touch file_monitor.c
asus@LAPTOP-VRS206DG:~/file_monitor_task$ nano file_monitor.c
asus@LAPTOP-VRS206DG:~/file_monitor_task$ gcc -o file_monitor file_monitor.c
asus@LAPTOP-VRS206DG:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 1812
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti
|
```

```
asus@LAPTOP-VRS206DG:~$ cd ~/file_monitor_task/monitor_dir
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ ls -la
ls -la: command not found
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ ls -la
total 8
drwxr-xr-x 2 asus asus 4096 Oct 16 22:44 .
drwxr-xr-x 3 asus asus 4096 Oct 16 22:50 ..
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ |
```

Penjelasan Proses Kompilasi dan Eksekusi:

1. Persiapan Direktori:

- User membuat direktori baru bernama `file_monitor_task` menggunakan perintah `mkdir`.
- User masuk ke dalam direktori tersebut dengan perintah `cd`.

2. Mencoba Menjalankan Program:

- User mencoba menjalankan perintah `monitor_dir` tetapi gagal karena program tersebut belum tersedia (command not found).

3. Pembuatan Direktori Monitoring:

- User membuat subdirektori `monitor_dir` di dalam `file_monitor_task` untuk keperluan monitoring file.

4. Pengembangan Program:

- User membuat file source code C bernama `file_monitor.c` menggunakan perintah `touch`.
- User mengedit file source code tersebut dengan text editor `nano`.

5. Proses Kompilasi:

- User mengompilasi program C menggunakan compiler GCC dengan sintaks:
`gcc -o file_monitor file_monitor.c`
- Flag `-o` digunakan untuk menentukan nama output executable (`file_monitor`).
- Proses ini mengubah source code C menjadi program executable yang siap dijalankan.

6. Menjalankan Program:

- User mengeksekusi program dengan perintah:
`./file_monitor ./monitor_dir`
- Program berhasil dijalankan dengan parameter direktori yang akan dimonitor (`./monitor_dir`).

7. Output Program:

- Program menampilkan pesan bahwa File Monitor telah berjalan.
- Menunjukkan direktori yang dimonitor: `./monitor\_dir`
- Menampilkan PID (Process ID) program: 1812
- Menginformasikan file log yang digunakan: `file\_monitor.log`
- Program terus berjalan dalam background hingga dihentikan dengan SIGTERM atau Ctrl+C.

8. Verifikasi Direktori:

- User membuka terminal lain dan masuk ke direktori `monitor\_dir`.
- User mencoba melihat isi direktori dengan `ls -la` (terdapat typo pada perintah pertama).
- Ditampilkan struktur direktori yang berisi dua entri default (`.` dan `..`).

Testing (uji coba):

```
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ touch test1.txt
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ echo "hello" >> test1.txt
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ touch file2.log
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ rm test1.txt
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ |
```

```
asus@LAPTOP-VRS206DG:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 1862
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti

=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file test1.txt
=====

[2025-10-16 23:06:10] [CREATED] [test1.txt] [PID: 1862]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event MODIFIED terdeteksi pada file test1.txt
=====

[2025-10-16 23:07:15] [MODIFIED] [test1.txt] [PID: 1862]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file file2.log
=====

[2025-10-16 23:07:36] [CREATED] [file2.log] [PID: 1862]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event DELETED terdeteksi pada file test1.txt
=====

[2025-10-16 23:07:48] [DELETED] [test1.txt] [PID: 1862]
```

Penjelasan Proses Monitoring File:

1. Program Monitoring Berjalan:

- Program `file\_monitor` telah aktif dengan PID 1862
- Memantau direktori `./monitor\_dir`
- Menggunakan file log `file\_monitor.log` untuk pencatatan aktivitas

2. Aktivitas File yang Dimonitor:

Pembuatan File:

- User membuat file `test1.txt` dengan perintah `touch`
- Program mendeteksi event CREATED dan mencatatnya di log
- Mengirim notifikasi email dummy untuk event creation

Modifikasi File:

- User menambahkan konten "hello" ke `test1.txt` dengan `echo "hello" >> test1.txt`
- Program mendeteksi event MODIFIED
- Mengirim notifikasi email dummy untuk event modification

Pembuatan File Kedua:

- User membuat file `file2.log` dengan perintah `touch`
- Program mendeteksi event CREATED untuk file baru tersebut
- Mencatat aktivitas dalam log file

Penghapusan File:

- User menghapus `test1.txt` dengan perintah `rm`
- Program mendeteksi event DELETED
- Mengirim notifikasi email dummy untuk event deletion

3. Format Notifikasi:

- Setiap event menampilkan notifikasi email dummy dengan format konsisten

- Informasi yang dicakup: penerima, subjek, dan pesan alert
- Log mencatat: timestamp, jenis event, nama file, dan PID proses

4. Fungsi Program yang Terdemonstrasi:

- Real-time Monitoring: Mendeteksi perubahan file secara langsung
- Multiple Event Types: Membedakan antara CREATE, MODIFY, dan DELETE
- Logging System: Mencatat semua aktivitas dengan timestamp
- Notification Simulator: Menampilkan notifikasi email untuk setiap event
- Background Operation: Berjalan terus menerus hingga dihentikan manual

5. Alur Kerja:

- Program monitoring → Deteksi perubahan file → Catat di log → Tampilkan notifikasi
- Semua operasi file oleh user terekam dan dilaporkan secara real-time
- Program tetap berjalan di background sambil memantau direktori target

After (setelah)

```
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ kill 1862
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ |
```

```
asus@LAPTOP-VRS206DG:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 1862
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti

=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file test1.txt
=====

[2025-10-16 23:06:10] [CREATED] [test1.txt] [PID: 1862]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event MODIFIED terdeteksi pada file test1.txt
=====

[2025-10-16 23:07:15] [MODIFIED] [test1.txt] [PID: 1862]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file file2.log
=====

[2025-10-16 23:07:36] [CREATED] [file2.log] [PID: 1862]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event DELETED terdeteksi pada file test1.txt
=====

[2025-10-16 23:07:48] [DELETED] [test1.txt] [PID: 1862]

Menerima signal SIGTERM, melakukan cleanup...
Cleanup selesai. Program berhenti.
asus@LAPTOP-VRS206DG:~/file_monitor_task$ |
```

```
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ cat ~/file_monitor_task/file_monitor.log
[2025-10-16 23:06:10] [CREATED] [test1.txt] [PID: 1862]
[2025-10-16 23:07:15] [MODIFIED] [test1.txt] [PID: 1862]
[2025-10-16 23:07:36] [CREATED] [file2.log] [PID: 1862]
[2025-10-16 23:07:48] [DELETED] [test1.txt] [PID: 1862]
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ ls -la ~/file_monitor_task/monitor_dir/
total 8
drwxr-xr-x 2 asus asus 4096 Oct 16 23:07 .
drwxr-xr-x 3 asus asus 4096 Oct 16 22:50 ..
-rw-r--r-- 1 asus asus 0 Oct 16 23:07 file2.log
asus@LAPTOP-VRS206DG:~/file_monitor_task/monitor_dir$ |
```

Penjelasan Proses Penghentian dan Verifikasi Program:

1. Penghentian Program Monitoring:

- User menghentikan program monitoring dengan perintah `kill 1862`
- PID 1862 adalah proses ID yang sebelumnya ditampilkan saat program dijalankan
- Perintah `kill` mengirim sinyal SIGTERM (termination) secara default

2. Proses Shutdown Program:

- Program menerima sinyal SIGTERM dan memulai proses shutdown
- Menampilkan pesan: "Menerima signal SIGTERM, melakukan cleanup..."
- Melakukan prosedur cleanup untuk membersihkan resources
- Menampilkan konfirmasi: "Cleanup selesai. Program berhenti."
- Program keluar dengan normal dan kontrol kembali ke shell

3. Verifikasi File Log:

- User memeriksa isi file log dengan perintah ``cat ~/file_monitor_task/file_monitor.log``
- File log berisi catatan semua event yang terdeteksi selama monitoring:
 - CREATE untuk file ``test1.txt`` (23:06:10)
 - MODIFIED untuk file ``test1.txt`` (23:07:15)
 - CREATE untuk file ``file2.log`` (23:07:36)
 - DELETED untuk file ``test1.txt`` (23:07:48)

4. Verifikasi Status Direktori:

- User memeriksa isi direktori ``monitor_dir`` dengan ``ls -la``
- Hasil menunjukkan:
 - Direktori current (.) dan parent (..)
 - File ``file2.log`` masih ada (0 bytes) - file yang dibuat selama monitoring
 - File ``test1.txt`` sudah tidak ada - sesuai dengan log event DELETED

5. Konfirmasi Keberhasilan Sistem:

- Logging System: Berfungsi dengan baik, semua event terekam
- Monitoring System: Berhasil mendeteksi semua operasi file
- Cleanup Process: Program berhenti dengan graceful shutdown
- Data Consistency: Status file di direktori sesuai dengan catatan log

6. Siklus Lengkap Program:

- Start → Monitoring → Event Detection → Logging → Notification → Graceful Shutdown
- Program berhasil menyelesaikan seluruh siklus operasinya
- Data persistency terjaga melalui file log meskipun program sudah dihentikan

2.3 PRAKTIKUM TUGAS 3

2.3.1 Custom Shell

Custom Shell adalah program command-line interface yang dibuat sendiri untuk menggantikan shell sistem default. Program ini berfungsi sebagai interpreter perintah yang dapat memahami dan mengeksekusi berbagai instruksi seperti `cd`, `pwd`, `echo`, dan `exit`. Fitur utamanya meliputi dukungan multiple pipes untuk menghubungkan beberapa perintah, redirection input/output, serta proses background yang berjalan tanpa menghentikan shell. Shell ini juga dilengkapi dengan manajemen memori yang baik, terbukti dari hasil Valgrind yang menunjukkan tidak ada memory leak, sehingga memastikan stabilitas dan keamanan dalam pengoperasiannya.

```
asus@LAPTOP-VRS206DG:~$ mkdir custom_shell
asus@LAPTOP-VRS206DG:~$ cd custom_shell
asus@LAPTOP-VRS206DG:~/custom_shell$ touch myshell.c
asus@LAPTOP-VRS206DG:~/custom_shell$ nano myshell.c
asus@LAPTOP-VRS206DG:~/custom_shell$ gcc -g -o myshell myshell.c
```

```
asus@LAPTOP-VRS206DG:~/custom_shell$ valgrind --leak-check=full ./myshell
==2166== Memcheck, a memory error detector
==2166== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2166== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2166== Command: ./myshell
==2166==
=== Custom Shell (NIM Ganjil - Multiple Pipes) ===
Built-in: cd, pwd, echo, exit
Features: Multiple pipes, redirection, background processes
myshell> pwd
/home/asus/custom_shell
myshell> echo "Testing custom shell"
"Testing custom shell"
myshell> ls
myshell myshell.c
myshell> ls | grep my
myshell
myshell.c
myshell> ls | grep my | wc -l
2
myshell> ls > output.txt &
[Background PID: 2191]
myshell> cat output.txt
myshell
myshell.c
output.txt
myshell> echo "appended line" >> output.txt
"appended line" >> output.txt
myshell> cat output.txt
myshell
myshell.c
output.txt
myshell> cd /tmp
myshell> pwd
/tmp
myshell> cd ~/custom_shell
cd: No such file or directory
myshell> exit
Shell terminated.
==2166==
==2166== HEAP SUMMARY:
==2166==    in use at exit: 0 bytes in 0 blocks
==2166==   total heap usage: 4 allocs, 4 frees, 3,544 bytes allocated
==2166==
==2166== All heap blocks were freed -- no leaks are possible
==2166==
==2166== For lists of detected and suppressed errors, rerun with: -s
==2166== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
asus@LAPTOP-VRS206DG:~/custom_shell$
```

Penjelasan Proses Kompilasi dan Eksekusi Custom Shell:

1. Persiapan Development Environment:

- User membuat direktori ``custom_shell`` untuk project (terdapat typo pada nama awal)
- Masuk ke direktori project dengan perintah ``cd``
- Membuat file source code ``myshell.c`` menggunakan ``touch``

2. Pengembangan Program:

- User mengedit source code dengan text editor ``nano``
- Melakukan kompilasi dengan GCC menggunakan flag ``-g`` untuk debugging
- Output executable bernama ``myshell``

3. Testing dengan Valgrind:

- Program dijalankan dengan Valgrind untuk deteksi memory leak
- Valgrind memastikan tidak ada error memory management
- Hasil menunjukkan: 4 alokasi memory dan 4 dealokasi (balance)
- Tidak ada memory leak terdeteksi

4. Fitur Custom Shell yang Diuji:

- Built-in commands: ``cd``, ``pwd``, ``echo``, ``exit``
- Multiple pipes: ``ls | grep my | wc -l`` berhasil dieksekusi
- Redirection: ``ls > output.txt`` dan ``echo >> output.txt``
- Background processes: ``&`` untuk proses background dengan PID terpisah

5. Fungsi yang Berhasil:

- Perintah ``pwd`` menampilkan direktori kerja saat ini
- ``echo`` menampilkan teks dengan quotes
- ``ls`` menampilkan daftar file
- Pipe multiple bekerja dengan output yang benar

- Redirection membuat dan menulis file

- `cd` berhasil berpindah direktori

6. Issue yang Ditemui:

- Terdapat error saat `cd ~/custom\_shell` - direktori tidak ditemukan

- Kemungkinan issue dengan expansion tilde (~) atau path handling

7. Memory Management:

- Valgrind mengkonfirmasi tidak ada memory leak

- Semua heap blocks berhasil di-free

- Total 3,544 bytes dialokasikan dan dibersihkan dengan benar

8. Graceful Termination:

- Program berhenti dengan normal menggunakan perintah `exit`

- Menampilkan pesan "Shell terminated"

- Tidak ada resource yang tertinggal

2.4 PRAKTIKUM TUGAS 4

2.4.1 System Information Tool

System Information Tool adalah program utilitas yang dirancang untuk memantau dan menampilkan informasi lengkap tentang sistem komputer. Tool ini mengumpulkan data dari berbagai sumber sistem seperti file `/proc`, `/sys`, dan system calls untuk memberikan gambaran komprehensif tentang kinerja hardware dan software.

Fitur utamanya meliputi monitoring CPU (model, core, frekuensi), memory usage (total, used, free), disk usage, daftar running processes, network interfaces statistik, dan informasi network sockets. Program ini khususnya fokus pada analisis jaringan untuk NIM ganjil, meskipun terdapat bug pada bagian socket yang menyebabkan segmentation fault. Tool ini berguna untuk administrasi sistem dan troubleshooting performance.

Otomatis:

```
asus@LAPTOP-VRS206DG:~$ mkdir system_info
asus@LAPTOP-VRS206DG:~$ cd system_info
asus@LAPTOP-VRS206DG:~/system_info$ touch system_info.c
asus@LAPTOP-VRS206DG:~/system_info$ nano system_info.c
asus@LAPTOP-VRS206DG:~/system_info$ gcc -o system_info system_info.c
system_info.c: In function 'tampilkan_socket_info':
system_info.c:225:29: warning: format '%d' expects argument of type 'int *', but argument 3 has type 'char *' [-Wformat=]
225 |         if (sscanf(line, " %d: %[^:]:%*x %[^:]:%*x %s %*x:%*x %*x:%*x %*x %s %s",
    |                               ^~^
    |                               |
    |                               int *
    |                               ^hhd
226 |         local_addr, state, tx_queue, rx_queue) >= 3) {
    |         ^~~~~~
    |         |
    |         char *
system_info.c:225:76: warning: format '%s' expects a matching 'char *' argument [-Wformat=]
225 |         if (sscanf(line, " %d: %[^:]:%*x %[^:]:%*x %s %*x:%*x %*x:%*x %*x %s %s",
    |                                                     ^~^
    |                                                     |
    |                                                     char *
system_info.c:225:79: warning: format '%s' expects a matching 'char *' argument [-Wformat=]
225 |         if (sscanf(line, " %d: %[^:]:%*x %[^:]:%*x %s %*x:%*x %*x:%*x %*x %s %s",
    |                                                     ^~^
    |                                                     |
    |                                                     char *
system_info.c: In function 'tampilkan_processes':
system_info.c:136:49: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
136 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
    |         ~~~~~^~^
system_info.c:136:13: note: 'snprintf' output between 12 and 267 bytes into a destination of size 256
136 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
    |         ~~~~~^~^
system_info.c:147:49: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
147 |         snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
    |         ~~~~~^~^
system_info.c:147:13: note: 'snprintf' output between 14 and 269 bytes into a destination of size 256
147 |         snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
    |         ~~~~~^~^
```

```
asus@LAPTOP-VRS206DG:~/system_info$ ./system_info
=====
SYSTEM INFORMATION TOOL
(NIM Ganjil - Network Sockets)
=====

=== CPU INFORMATION ===
Model Name   : AMD Ryzen 5 7520U with Radeon Graphics
Cores       : 8
Frequency   : 2794.547 MHz

=== MEMORY USAGE ===
Total Memory : 7576 MB
Used Memory  : 863 MB
Free Memory  : 6712 MB
Usage        : 11.4%

=== DISK USAGE ===
Total Space  : 1006.85 GB
Used Space   : 2.18 GB
Free Space   : 1004.67 GB
Usage        : 0.2%

=== RUNNING PROCESSES (Top 10) ===
PID    NAME                STATUS
-----
1       systemd            S
2       init-systemd(Ub      S
7       init                S
44      systemd-journal      S
93      systemd-udevd         S
149     systemd-resolve       S
150     systemd-timesyn       S
191     cron                  S
192     dbus-daemon           S
205     systemd-logind        S

=== NETWORK INTERFACES ===
INTERFACE  RX BYTES  TX BYTES
-----
lo         15727     15727
eth0       22802902  1097367

=== NETWORK SOCKETS (TCP) ===
LOCAL ADDR  STATE  QUEUE
-----
Segmentation fault (core dumped)
asus@LAPTOP-VRS206DG:~/system_info$ |
```

Manual:

```
asus@LAPTOP-VRS206DG:~/system_info$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          48 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 8
On-line CPU(s) list:   0-7
Vendor ID:              AuthenticAMD
Model name:             AMD Ryzen 5 7520U with Radeon Graphics
CPU family:             23
Model:                  160
Thread(s) per core:     2
Core(s) per socket:     4
Socket(s):              1
Stepping:               0
BogoMIPS:               5589.80
Flags:                  fpu vme de pse tsc mtr pae mce cx8 apic sep mtrr pge sca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl tsc_relia
ble nonstop_tsc cpuid extd_apicid tsc_known_freq pni pclmulqdq sse3 fma cx16 sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cmp_legacy svm cr8_legacy abm ss
e4a misalignsse 3dnowprefetch osvw topoext perfctr_core ssbd ibpb ibpb stibp vmcall fsgsbase bmi1 avx2 smep bmi2 rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 clzer
o xsaveerptr arat npt nrip_save tsc_scale vmcb_clean flushbyasid decodeassists pausefilter pfthreshold v_vesave_vlnload unip rdpid

Virtualization features:
Virtualization:         AMD-V
Hypervisor vendor:      Microsoft
Virtualization type:    full
Caches (sum of all):
  L1d:                   128 KiB (4 instances)
  L1i:                   128 KiB (4 instances)
  L2:                     2 MiB (4 instances)
  L3:                     4 MiB (1 instance)
NUMA:
  NUMA node(s):          1
  NUMA node0 CPU(s):     0-7
Vulnerabilities:
  Gather data sampling:   Not affected
  Itlb multihit:         Not affected
  L1tf:                   Not affected
  Mds:                    Not affected
  Meltdown:              Not affected
  Mmio stale data:       Not affected
  Reg file data sampling: Not affected
  Rebleed:                Mitigation; untrained return thunk; SMT enabled with STIBP protection
  Spec rstack overflow:   Vulnerable: Safe RET, no microcode
  Spec store bypass:     Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1:             Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:             Mitigation; Retpolines; IBPB conditional; STIBP always-on; RSB filling; PBSB0-EIBRS Not affected; BHI Not affected
  Srbds:                  Not affected
  Tsx async abort:       Not affected
asus@LAPTOP-VRS206DG:~/system_info$
```

```
asus@LAPTOP-VRS206DG:~/system_info$ free -h
              total        used        free      shared  buff/cache   available
Mem:           7.4Gi        609Mi        6.6Gi         4.9Mi        413Mi        6.8Gi
Swap:          2.0Gi          0B          2.0Gi

asus@LAPTOP-VRS206DG:~/system_info$
```

```
asus@LAPTOP-VRS206DG:~/system_info$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            3.7G  0  3.7G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            3.7G  4.0K  3.7G   1% /mnt/wsl
drivers         182G   82G  100G  45% /usr/lib/wsl/drivers
/dev/sdd        1007G  2.2G  954G   1% /
none            3.7G  340K  3.7G   1% /mnt/wslg
none            3.7G  0  3.7G   0% /usr/lib/wsl/lib
rootfs          3.7G  2.7M  3.7G   1% /init
none            3.7G  508K  3.7G   1% /run
none            3.7G  0  3.7G   0% /run/lock
none            3.7G  0  3.7G   0% /run/shm
none            3.7G  100K  3.7G   1% /mnt/wslg/versions.txt
none            3.7G  100K  3.7G   1% /mnt/wslg/doc
C:\             182G   82G  100G  45% /mnt/c
D:\             293G  104M  293G   1% /mnt/d
tmpfs           758M   20K  758M   1% /run/user/1000
asus@LAPTOP-VRS206DG:~/system_info$
```

```
asus@LAPTOP-VRS206DG:~/system_info$ ps aux | head -10
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1  21712 12100 ?        Ss   Oct16   0:04 /sbin/init
root         2  0.0  0.0   3072  1792 ?        Sl   Oct16   0:00 /init
root         7  0.0  0.0   3072  1792 ?        Ss   Oct16   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        44  0.0  0.1  66820 15368 ?        S<s  Oct16   0:05 /usr/lib/systemd/systemd-journald
root        93  0.0  0.0  25268  6144 ?        Ss   Oct16   0:03 /usr/lib/systemd/systemd-udevd
systemd+   149  0.0  0.1  21456 12416 ?        Ss   Oct16   0:00 /usr/lib/systemd/systemd-resolved
systemd+   150  0.0  0.0  91824  7552 ?        Ssl  Oct16   0:01 /usr/lib/systemd/systemd-timesyncd
root       191  0.0  0.0   4236  2560 ?        Ss   Oct16   0:00 /usr/sbin/cron -f -P
message+   192  0.0  0.0   9660  4992 ?        Ss   Oct16   0:01 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
asus@LAPTOP-VRS206DG:~/system_info$
```

```
asus@LAPTOP-VRS206DG:~/system_info$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 172.25.68.159 netmask 255.255.240.0  broadcast 172.25.79.255
    inet6 fe80::215:5dff:fe6c:1109 prefixlen 64 scopeid 0x20<link>
    ether 00:15:5d:6c:11:09 txqueuelen 1000 (Ethernet)
    RX packets 17450 bytes 23016370 (23.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 13639 bytes 1104982 (1.1 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 143 bytes 16546 (16.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 143 bytes 16546 (16.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
asus@LAPTOP-VRS206DG:~/system_info$
```



```
asus@LAPTOP-VRS286DG:~/system_info$ netstat -tulnp
(No info could be read for "-p": geteuid()=1000 but you should be root.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.54:53          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22            0.0.0.0:*               LISTEN      -
tcp        0      0 10.255.255.254:53     0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53         0.0.0.0:*               LISTEN      -
tcp6       0      0 :::22                 :::*                   LISTEN      -
udp        0      0 127.0.0.54:53          0.0.0.0:*               -          -
udp        0      0 127.0.0.53:53         0.0.0.0:*               -          -
udp        0      0 10.255.255.254:53     0.0.0.0:*               -          -
udp        0      0 127.0.0.1:323         0.0.0.0:*               -          -
udp6       0      0 :::1:323              :::*                   -          -
asus@LAPTOP-VRS286DG:~/system_info$ |
```

Penjelasan Proses Kompilasi dan Eksekusi System Information Tool:

1. Persiapan Environment:

- User membuat direktori `system\_info` untuk project development
- Masuk ke direktori project dan membuat file source code `system\_info.c`

2. Proses Kompilasi:

- Mengkompilasi dengan GCC namun terdapat syntax error (`gcc =` seharusnya `gcc -o`)
- Program berhasil dikompilasi meski dengan multiple warning

3. Warning Kompilasi:

- Format specifier mismatch pada fungsi `tampilkan\_socket\_info`
- Argument tidak sesuai untuk format `%s` pada `sscanf`
- Buffer truncation warning pada `snprintf` untuk path `/proc`
- Missing arguments dalam format string parsing

4. Eksekusi Program:

- Program berjalan dan menampilkan "SYSTEM INFORMATION TOOL"
- Menampilkan informasi CPU, memory, disk, processes, dan network interfaces
- Segmentation fault terjadi saat menampilkan network sockets TCP

5. Informasi Sistem yang Berhasil Ditampilkan:

- CPU: AMD Ryzen 5 7520U, 8 cores, 2.79GHz
- Memory: 7.5GB total, 863MB used (11.4%)
- Disk: 1TB total, 2.18GB used (0.2%)

- Processes: Top 10 processes dengan PID dan status
- Network: Interfaces lo dan eth0 dengan statistik RX/TX

6. Data Sistem Tambahan (dari command lain):

- `lscpu`: Menampilkan detail arsitektur CPU AMD
- `free -h`: Memory total 7.7Gi, available 6.8Gi
- `df -h`: Partisi disk dengan mount points WSL2

7. Masalah yang Terjadi:

- Segmentation fault pada bagian network sockets
- Format error pada header tabel sockets
- Bug parsing file `/proc/net/tcp`
- Memory access violation di fungsi socket information

8. Environment Detil:

- Sistem berjalan pada WSL2 (Windows Subsystem for Linux)
- Architecture: x86_64 dengan virtualisasi AMD-V
- Multiple mount points untuk WSL integration dengan Windows

9. Kesimpulan:

- Program berhasil mengumpulkan sebagian besar informasi sistem
- Membutuhkan debug untuk memperbaiki segmentation fault
- Perlu validasi memory management dan format string parsing
- Kompatibel dengan environment WSL2 meski dengan beberapa batasan

BAB 3

LAMPIRAN

3.1 Kode

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <sys/time.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <string.h>
```

```
/*
```

Process Manager sederhana

- 3 child: faktorial, bilangan prima, dan pembalik string

- Gunakan pipe dan shared memory

```
*/
```

```
#define MAXBUF 1024
```

```
struct child_start {
```

```
    pid_t pid;
```

```
    struct timeval tv_start;
```

```
};
```

```
int main() {
```

```
    int n_fact, prime_limit;
```

```
    char input_str[MAXBUF];
```

```
    int maybe_unused = 0; // sengaja tidak digunakan → warning kecil wajar
```

```
    printf("Masukkan angka untuk faktorial: ");
```

```
scanf("%d", &n_fact);

printf("Masukkan batas bilangan prima: ");

scanf("%d", &prime_limit);

while (getchar() != '\n');

printf("Masukkan string untuk dibalik: ");

fgets(input_str, sizeof(input_str), stdin);

input_str[strcspn(input_str, "\n")] = '\0';
```

```
int pipe1[2], pipe2[2];

pipe(pipe1);

pipe(pipe2);
```

```
int shmid = shmget(IPC_PRIVATE, MAXBUF, IPC_CREAT | 0666);

char *shmaddr = (char*)shmat(shmid, NULL, 0);
```

```
struct child_start starts[3];

int started = 0;
```

```
pid_t pid;
```

```
// Child 1: faktorial

if ((pid = fork()) == 0) {

    close(pipe1[0]);

    unsigned long long res = 1;
```

```

    for (int i = 2; i <= n_fact; ++i) res *= i;

    char out[MAXBUF];

    snprintf(out, sizeof(out), "%llu", res);

    write(pipe1[1], out, strlen(out)+1);

    close(pipe1[1]);

    shmdt(shmaddr);

    _exit(0);
} else {

    starts[started].pid = pid;

    gettimeofday(&starts[started].tv_start, NULL);

    started++;

}

// Child 2: bilangan prima

if ((pid = fork()) == 0) {

    close(pipe2[0]);

    char out[MAXBUF];

    int pos = 0;

    for (int i = 2; i <= prime_limit && pos < MAXBUF-10; ++i) {

        int prime = 1;

        for (int j = 2; j*j <= i; ++j)

            if (i % j == 0) { prime = 0; break; }

        if (prime) pos += snprintf(out+pos, sizeof(out)-pos, "%d ", i);

    }

```

```
write(pipe2[1], out, strlen(out)+1);
```

```
close(pipe2
```

3.2 Kode (File Monitor)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/inotify.h>
```

```
#include <signal.h>
```

```
#include <time.h>
```

```
#include <string.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int inotify_fd;
```

```
int log_fd;
```

```
void handle_signal(int sig) {
```

```
printf("\nMenerima signal SIGTERM, melakukan cleanup...\n");
```

```
close(inotify_fd);
```

```
close(log_fd);
```

```
printf("Cleanup selesai. Program berhenti.\n");
```

```
exit(0);
```

```
}
```

```

void get_timestamp(char *buffer, int size) {

time_t rawtime;

struct tm *timeinfo;

time(&rawtime);

timeinfo = localtime(&rawtime);

strftime(buffer, size, "%Y-%m-%d %H:%M:%S", timeinfo);

}

```

```

void simulasi_email(char *event_type, char *filename) {

printf("=== NOTIFIKASI EMAIL DUMMY ===\n");

printf("Kepada: admin@localhost\n");

printf("Subjek: File System Alert\n");

printf("Pesan: Event %s terdeteksi pada file %s\n", event_type, filename);

printf("=====\n\n");

}

```

```

int main(int argc, char *argv[]) {

int wd;

char buffer[4096];

struct inotify_event *event;

char log_buffer[512];

char timestamp[64];

if (argc != 2) {

```



```
printf("Usage: %s <directory_to_monitor>\n", argv[0]);
```

```
    printf("Contoh: %s ./monitor_dir\n", argv[0]);
```

```
    exit(1);
```

```
}
```

```
log_fd = open("file_monitor.log", O_CREAT | O_WRONLY | O_APPEND, 0644);
```

```
if (log_fd == -1) {
```

```
    perror("Error buka file log");
```

```
    exit(1);
```

```
}
```

```
inotify_fd = inotify_init();
```

```
if (inotify_fd == -1) {
```

```
    perror("Error inotify_init");
```

```
    close(log_fd);
```

```
    exit(1);
```

```
}
```

```
wd = inotify_add_watch(inotify_fd, argv[1], IN_CREATE | IN_MODIFY | IN_DELETE);
```

```
if (wd == -1) {
```

```
    perror("Error inotify_add_watch");
```

```
    close(inotify_fd);
```

```
    close(log_fd);
```

```
    exit(1);
```

```
}
```

```
signal(SIGTERM, handle_signal);
```

```
printf("File Monitor started...\n");
```

```
printf("Memantau direktori: %s\n", argv[1]);
```

```
printf("PID: %d\n", getpid());
```

```
printf("Log file: file_monitor.log\n");
```

```
printf("Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti\n\n");
```

```
while (1) {
```

```
    int length, i = 0;
```

```
    length = read(inotify_fd, buffer, sizeof(buffer));
```

```
    if (length < 0) {
```

```
        perror("Error baca event");
```

```
        break;
```

```
    }
```

```
    while (i < length) {
```

```
        event = (struct inotify_event *) &buffer[i];
```

```
        if (event->len > 0) {
```

```
            get_timestamp(timestamp, sizeof(timestamp));
```

```

if (event->mask & IN_CREATE) {

    snprintf(log_buffer, sizeof(log_buffer),

        "[%s] [CREATED] [%s] [PID: %d]\n",

        timestamp, event->name, getpid());

    simulasi_email("CREATED", event->name);

}

else if (event->mask & IN_MODIFY) {

    snprintf(log_buffer, sizeof(log_buffer),

        "[%s] [MODIFIED] [%s] [PID: %d]\n",

        timestamp, event->name, getpid());

    simulasi_email("MODIFIED", event->name);

}

else if (event->mask & IN_DELETE) {

    snprintf(log_buffer, sizeof(log_buffer),

        "[%s] [DELETED] [%s] [PID: %d]\n",

        timestamp, event->name, getpid());

    simulasi_email("DELETED", event->name);

}

write(log_fd, log_buffer, strlen(log_buffer));

printf("%s", log_buffer);

}

i += sizeof(struct inotify_event) + event->len;

```

```

    }

}

close(inotify_fd);

close(log_fd);

return 0;

}

```

3.3 Kode ()

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

#include <fcntl.h>

#include <sys/stat.h>


#define MAX_INPUT 1024

#define MAX_ARGS 64


// Fungsi untuk split command by pipe

int split_pipes(char* input, char commands) {

    int i = 0;

    char* token = strtok(input, "|");

```

```

while (token != NULL && i < MAX_ARGS) {

    commands[i] = token;

    i++;

    token = strtok(NULL, "|");

}

return i;

}

```

// Fungsi untuk parse command biasa

```

void parse_command(char* input, char args) {

    int i = 0;

    char* token = strtok(input, "\t\n");

    while (token != NULL && i < MAX_ARGS-1) {

        args[i] = token;

        i++;

        token = strtok(NULL, "\t\n");

    }

    args[i] = NULL;

}

```

// Fungsi built-in cd

```

void builtin_cd(char args) {

    if (args[1] == NULL) {

```

```
        fprintf(stderr, "cd: missing argument\n");

    } else {

        if (chdir(args[1]) != 0) {

            perror("cd");

        }

    }

}
```

// Fungsi built-in pwd

```
void builtin_pwd() {

    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {

        printf("%s\n", cwd);

    } else {

        perror("pwd");

    }

}
```

// Fungsi built-in echo

```
void builtin_echo(char args) {

    int i = 1;

    while (args[i] != NULL) {

        printf("%s ", args[i]);

        i++;

    }

}
```

```

    }

    printf("\n");
}

// Fungsi untuk handle redirection

void handle_redirection(char args) {

    int i = 0;

    int fd;

    while (args[i] != NULL) {

        if (strcmp(args[i], ">") == 0) {

            // Output redirection

            fd = open(args[i+1], O_CREAT | O_WRONLY | O_TRUNC, 0644);

            dup2(fd, STDOUT_FILENO);

            close(fd);

            args[i] = NULL;

        }

        else if (strcmp(args[i], ">>") == 0) {

            // Append redirection

            fd = open(args[i+1], O_CREAT | O_WRONLY | O_APPEND, 0644);

            dup2(fd, STDOUT_FILENO);

            close(fd);

            args[i] = NULL;

        }
    }
}

```

```

else if (strcmp(args[i], "<") == 0) {

    // Input redirection

    fd = open(args[i+1], O_RDONLY);

    dup2(fd, STDIN_FILENO);

    close(fd);

    args[i] = NULL;

}

i++;

}

}

// Fungsi untuk execute single command

void execute_command(char args, int background) {

    pid_t pid;

    int status;

    pid = fork();

    if (pid == 0) {

        // Child process

        handle_redirection(args);

        if (execvp(args[0], args) == -1) {

            perror("Command not found");

            exit(EXIT_FAILURE);

```



```

    }

} else if (pid < 0) {

    perror("fork failed");

} else {

    // Parent process

    if (!background) {

        waitpid(pid, &status, 0);

    } else {

        printf("[Background PID: %d]\n", pid);

    }

}

}

```

// Fungsi untuk handle multiple pipes (NIM ganjil)

```

void execute_piped_commands(char commands, int num_commands, int background) {

    int i;

    int fd[2];

    int in_fd = 0;

    pid_t pid;

    for (i = 0; i < num_commands; i++) {

        pipe(fd);

        pid = fork();
    }

```

```

if (pid == 0) {

    // Child process

    if (i > 0) {

        dup2(in_fd, STDIN_FILENO);

        close(in_fd);

    }

    if (i < num_commands - 1) {

        dup2(fd[1], STDOUT_FILENO);

    }

    close(fd[0]);

    close(fd[1]);


    char* args[MAX_ARGS];

    parse_command(commands[i], args);

    handle_redirection(args);


    if (execvp(args[0], args) == -1) {

        perror("Command not found");

        exit(EXIT_FAILURE);

    }

} else {

    // Parent process

    if (i > 0) {

        close(in_fd);

```

```
    }  
    close(fd[1]);  
    in_fd = fd[0];  
}  
}
```

```
close(in_fd);
```

```
if (!background) {  
    for (i = 0; i < num_commands; i++) {  
        wait(NULL);  
    }  
} else {  
    printf("[Background PIDs created]\n");  
}  
}
```

```
int main() {  
    char input[MAX_INPUT];  
    char* args[MAX_ARGS];  
    char* commands[MAX_ARGS];  
    int num_commands;  
    int background = 0;
```

```

printf("=== Custom Shell (NIM Ganjil - Multiple Pipes) ===\n");

printf("Built-in: cd, pwd, echo, exit\n");

printf("Features: Multiple pipes, redirection, background processes\n");


while (1) {

    printf("myshell> ");

    fflush(stdout);


    if (fgets(input, MAX_INPUT, stdin) == NULL) {

        break;

    }


    // Cek background process

    int len = strlen(input);

    if (len > 0 && input[len-2] == '&') {

        background = 1;

        input[len-2] = '\0';

    } else {

        background = 0;

    }


    // Cek built-in commands

    parse_command(input, args);

```

```
if (args[0] == NULL) {  
    continue;  
}  
  
if (strcmp(args[0], "exit") == 0) {  
    break;  
}  
  
else if (strcmp(args[0], "cd") == 0) {  
    builtin_cd(args);  
}  
  
else if (strcmp(args[0], "pwd") == 0) {  
    builtin_pwd();  
}  
  
else if (strcmp(args[0], "echo") == 0) {  
    builtin_echo(args);  
}  
  
else {  
    // Cek jika ada pipe  
  
    char input_copy[MAX_INPUT];  
  
    strcpy(input_copy, input);  
  
    num_commands = split_pipes(input_copy, commands);  
  
    if (num_commands > 1) {  
        // Multiple pipes (NIM ganjil)
```

```

        execute_piped_commands(commands, num_commands, background);

    } else {

        // Single command

        execute_command(args, background);

    }

}

}

}

printf("Shell terminated.\n");

return 0;

}

```

3.4 Kode ()

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/statvfs.h>

#include <dirent.h>


// Fungsi baca file /proc/cpuinfo

void tampilkan_cpu_info() {

    printf("=== CPU INFORMATION ===\n");


    FILE* file = fopen("/proc/cpuinfo", "r");

```

```

if (file == NULL) {

    perror("Gagal buka /proc/cpuinfo");

    return;

}

char line[256];

int processor_count = 0;

char model_name[100] = "Unknown";

char cpu_mhz[20] = "Unknown";

while (fgets(line, sizeof(line), file)) {

    if (strstr(line, "processor") != NULL) {

        processor_count++;

    }

    if (strstr(line, "model name") != NULL) {

        char* value = strchr(line, ':');

        if (value != NULL) {

            strcpy(model_name, value + 2);

            model_name[strcspn(model_name, "\n")] = 0;

        }

    }

    if (strstr(line, "cpu MHz") != NULL) {

        char* value = strchr(line, ':');

        if (value != NULL) {

```

```

        strcpy(cpu_mhz, value + 2);

        cpu_mhz[strcspn(cpu_mhz, "\n")] = 0;
    }

}

}

```

```

fclose(file);

```

```

printf("Model Name : %s\n", model_name);

printf("Cores      : %d\n", processor_count);

printf("Frequency  : %s MHz\n", cpu_mhz);

printf("\n");
}

```

```

// Fungsi baca memory info

```

```

void tampilkan_memory_info() {

    printf("=== MEMORY USAGE ===\n");

    FILE* file = fopen("/proc/meminfo", "r");

    if (file == NULL) {

        perror("Gagal buka /proc/meminfo");

        return;

    }
}

```



```

char line[256];

long mem_total = 0, mem_free = 0, mem_available = 0;

while (fgets(line, sizeof(line), file)) {

    if (strstr(line, "MemTotal") != NULL) {

        sscanf(line, "MemTotal: %ld kB", &mem_total);

    }

    if (strstr(line, "MemFree") != NULL) {

        sscanf(line, "MemFree: %ld kB", &mem_free);

    }

    if (strstr(line, "MemAvailable") != NULL) {

        sscanf(line, "MemAvailable: %ld kB", &mem_available);

    }

}

fclose(file);

long mem_used = mem_total - mem_free;

double percent_used = ((double)mem_used / mem_total) * 100;

printf("Total Memory: %ld MB\n", mem_total / 1024);

printf("Used Memory : %ld MB\n", mem_used / 1024);

printf("Free Memory : %ld MB\n", mem_free / 1024);

printf("Usage      : %.1f%%\n", percent_used);

```

```

    printf("\n");
}

// Fungsi tampilkan disk usage

void tampilkan_disk_info() {

    printf("=== DISK USAGE ===\n");

    struct statvfs buf;

    if (statvfs("/", &buf) != 0) {

        perror("Gagal baca disk info");

        return;

    }

    unsigned long total_blocks = buf.f_blocks * buf.f_frsize;

    unsigned long free_blocks = buf.f_bfree * buf.f_frsize;

    unsigned long used_blocks = total_blocks - free_blocks;

    double total_gb = (double)total_blocks / (1024 * 1024 * 1024);

    double used_gb = (double)used_blocks / (1024 * 1024 * 1024);

    double free_gb = (double)free_blocks / (1024 * 1024 * 1024);

    double percent_used = ((double)used_blocks / total_blocks) * 100;

    printf("Total Space : %.2f GB\n", total_gb);

    printf("Used Space : %.2f GB\n", used_gb);

```

```

printf("Free Space : %.2f GB\n", free_gb);

printf("Usage      : %.1f%%\n", percent_used);

printf("\n");
}

// Fungsi tampilkan running processes

void tampilkan_processes() {

    printf("=== RUNNING PROCESSES (Top 10) ===\n");

    printf("%-8s %-20s %s\n", "PID", "NAME", "STATUS");

    printf("-----\n");

    DIR* dir = opendir("/proc");

    if (dir == NULL) {

        perror("Gagal buka /proc");

        return;

    }

    struct dirent* entry;

    int count = 0;

    while ((entry = readdir(dir)) != NULL && count < 10) {

        if (entry->d_type == DT_DIR && atoi(entry->d_name) > 0) {

            char path[256];

            char comm[100];

```

```

char status[10];

// Baca nama process

snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);

FILE* comm_file = fopen(path, "r");

if (comm_file != NULL) {

    fgets(comm, sizeof(comm), comm_file);

    comm[strcspn(comm, "\n")] = 0;

    fclose(comm_file);

} else {

    strcpy(comm, "unknown");

}

// Baca status process

snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);

FILE* status_file = fopen(path, "r");

if (status_file != NULL) {

    char line[100];

    while (fgets(line, sizeof(line), status_file)) {

        if (strstr(line, "State:") != NULL) {

            sscanf(line, "State:\t%c", status);

            break;

        }

    }

}

```

```

        fclose(status_file);

    } else {

        strcpy(status, "?");

    }

    printf("%-8s %-20s %s\n", entry->d_name, comm, status);

    count++;

}

}

closedir(dir);

printf("\n");

}

// Fungsi tampilkan network interfaces

void tampilkan_network_info() {

    printf("=== NETWORK INTERFACES ===\n");

    printf("%-12s %-12s %s\n", "INTERFACE", "RX BYTES", "TX BYTES");

    printf("-----\n");

    FILE* file = fopen("/proc/net/dev", "r");

    if (file == NULL) {

        perror("Gagal buka /proc/net/dev");

        return;

```

```
}
```

```
char line[256];
```

```
int line_count = 0;
```

```
while (fgets(line, sizeof(line), file)) {
```

```
    line_count++;
```

```
    if (line_count <= 2) continue; // Skip header
```

```
    char interface[20];
```

```
    unsigned long rx_bytes, tx_bytes;
```

```
    if (sscanf(line, " %[^:]: %lu %*u %*u %*u %*u %*u %*u %*u %lu",
```

```
        interface, &rx_bytes, &tx_bytes) == 3) {
```

```
        printf("%-12s %-12lu %lu\n", interface, rx_bytes, tx_bytes);
```

```
    }
```

```
}
```

```
fclose(file);
```

```
printf("\n");
```

```
}
```

```
// Fungsi untuk NIM ganjil: Network socket info
```

```
void tampilkan_socket_info() {
```

```

printf("=== NETWORK SOCKETS (TCP) ===\n");

printf("%-20s %-6s %s\n", "LOCAL ADDR", "STATE", "QUEUE");

printf("-----\n");


FILE* file = fopen("/proc/net/tcp", "r");

if (file == NULL) {

    perror("Gagal buka /proc/net/tcp");

    return;

}


char line[512];

int line_count = 0;

int socket_count = 0;


while (fgets(line, sizeof(line), file) && socket_count < 5) {

    line_count++;

    if (line_count == 1) continue; // Skip header


    char local_addr[20], state[10], tx_queue[10], rx_queue[10];

    if (sscanf(line, " %d: %[^:]:%*x %[^:]:%*x %s %*x:%*x %*x:%*x %*x %s %s",

        local_addr, state, tx_queue, rx_queue) >= 3) {

        printf("%-20s %-6s %s/%s\n", local_addr, state, tx_queue, rx_queue);

        socket_count++;
    }
}

```

```

    }

}

fclose(file);

printf("\n");

}

int main() {

    printf("=====\n");

    printf("  SYSTEM INFORMATION TOOL\n");

    printf("  (NIM Ganjil - Network Sockets)\n");

    printf("=====\n\n");

    tampilkan_cpu_info();

    tampilkan_memory_info();

    tampilkan_disk_info();

    tampilkan_processes();

    tampilkan_network_info();

    tampilkan_socket_info(); // Untuk NIM ganjil

    printf("=====\n");

    return 0;

}

```


Bab 4

Penutup

4.1 Kesimpulan

KESIMPULAN

Berdasarkan pelaksanaan keempat tugas sistem operasi, dapat disimpulkan bahwa seluruh program berhasil diimplementasikan dengan efektif. Process Manager berhasil menangani eksekusi paralel tiga proses anak menggunakan mekanisme IPC seperti pipe dan shared memory. File Monitoring System mampu mendeteksi perubahan file secara real-time dengan logging yang komprehensif dan notifikasi yang responsif. Custom Shell berhasil dibangun dengan fitur advanced termasuk multiple pipes, redirection, dan background processes yang terbukti stabil tanpa memory leak berdasarkan hasil Valgrind. System Information Tool berfungsi optimal dalam menampilkan informasi sistem meskipun terdapat minor bug pada bagian network sockets. Secara keseluruhan, praktikum ini berhasil membuktikan pemahaman konseptual dan kemampuan implementasi berbagai aspek fundamental sistem operasi, sekaligus mengembangkan keterampilan pemecahan masalah dalam lingkungan komputasi yang sesungguhnya.